TECHNICAL DESIGN DOCUMENT | SYSTEM SPECIFICATION

AURELIA MOTORS

Full-Stack Digital Dealership Platform & Multi-Facility Operations System

Project Attribute	Specification Detail
Project Name	Aurelia Motors Enterprise Dealership Platform
Author	Lochan T N
Project Nature	Software Engineering Hiring Project
Document Version	Version 1.0
Project Status	Completed Hiring Assignment
Automated Test Verification	127 Passing Automated Tests across 18 Test Suites (100% Pass Rate)
Target Network Topology	3 Showrooms & 2 Authorized Workshops in Lucknow, UP, India
Core Architecture	TypeScript Monorepo • React 19 SPA • Express 5 REST API • PostgreSQL 18 / 16+ • Prisma 6 ORM
Repository URL	https://github.com/lochan87/aurelia-motors
Date of Completion	November 2026

Executive Summary Note: This engineering design document provides an architectural and operational overview of the Aurelia Motors platform. It covers system topology, database schemas, core retail workflows, CRM routing algorithms, grounded domain intelligence, RBAC security, data privacy, and testing verification. This project represents an individual hiring assessment prototype, not a commercial production deployment.

Table of Contents

This document is organized into thirteen technical sections covering the complete system lifecycle:

- **1. Executive Summary:** High-level overview, physical network topology, and key architectural metrics
- **2. Problem Statement and Scope:** Franchise dealership challenges, DPDP Act 2023 compliance, Indian tax structure
- **3. Technology Stack and Architecture Decisions:** React 19, Express 5, PostgreSQL 18/16+, TypeScript, integer paise math standard
- **4. System Architecture Diagram:** Multi-tier containerized topology, edge gateway routing, and request lifecycle
- **5. Database Design and Entity Relationships:** Core ER model, Prisma schemas, facility scoping, and table catalog
- **6. Core Customer Workflows:** Discovery, on-road pricing formulas, capacity-safe booking, and trade-ins
- **7. Dealership Operations:** Lead routing, 30-min SLA engine, CSV portal ingestion, inventory CMS, price approvals
- **8. AI-Assisted Features and Limitations:** Aurelia Grounded Domain Engine, lead summaries, message drafts, boundaries
- **9. Authentication, Authorization and Data Privacy:** Argon2id, HttpOnly sessions, CSRF, RBAC matrix, DPDP 2023 compliance
- **10. Testing and Verification:** 127 automated tests across 18 suites, unit, integration, and journey verification
- **11. Local Setup and Deployment:** Prerequisites, local development, Docker Compose topology, demo credentials
- **12. Known Limitations and Future Improvements:** Simulated gateways, concurrency limits, production hardening roadmap
- **13. Repository and Reviewer Access:** GitHub repository details, test execution commands, and document governance

> **Architectural Focus: Operational Integrity & Determinism**
> Aurelia Motors prioritizes data integrity and determinism over speculative complexity. All monetary values are calculated in 64-bit integer paise to prevent floating-point rounding drift. Demonstrator fleet test-drives and workshop service bays enforce capacity checks using interactive transactions and pessimistic row-level database locks to serialize concurrent requests. The AI domain engine operates on grounded database heuristics rather than probabilistic cloud LLMs.

1. Executive Summary

Aurelia Motors is an individual software engineering hiring project designed to model an authorized multi-facility passenger car franchise network in Lucknow, Uttar Pradesh, India. The application unifies customer-facing digital automotive retail with internal back-office dealership operations.

The physical franchise network modeled and seeded in the platform comprises three primary retail showrooms and two dedicated authorized service workshops strategically positioned across Lucknow's commercial corridors:

- **Showroom 1: Central Lucknow (LKO-CENTRAL):** Flagship sales facility on Ashok Marg, Hazratganj, handling high-volume passenger sedan and crossover sales.
- **Showroom 2: Gomti Nagar (LKO-GOMTI):** Boutique digital retail centre situated in Gomti Nagar Phase 2, specializing in premium crossovers and executive electric sedans.
- **Showroom 3: Kanpur Road (LKO-KANPUR):** Commercial highway showroom located along the Kanpur-Lucknow highway corridor, handling utility vehicles and certified pre-owned units.
- **Workshop 1: Transport Nagar Workshop (LKO-WS-TRANS):** Heavy maintenance, accident repair, paint shop, and scheduled multi-bay vehicle servicing center with 8 service bays.
- **Workshop 2: Alambagh Workshop (LKO-WS-ALAM):** Fast-track periodic maintenance, express oil clinics, and diagnostic servicing workshop with 8 service bays.

The application provides public customer engagement—including vehicle model discovery, variant comparisons, transparent on-road pricing with reducing-balance EMI calculators, capacity-safe test-drive reservations, service appointment scheduling, and certified used-car browsing with 150-point inspection findings—alongside staff operations. Operational capabilities include role-based staff authentication, automated lead assignment with a 30-minute response-time SLA clock, external auto portal CSV batch ingestion, centralized CMS inventory management, two-person used-car price adjustments, and grounded domain intelligence.

The platform is implemented in a modern TypeScript monorepo using React 19, Express 5, and PostgreSQL (PostgreSQL 18 Alpine in Docker Compose, compatible with PostgreSQL 16+) via Prisma 6 ORM. The implementation is verified by 127 automated tests across 18 test suites and containerized via Docker Compose.

2. Problem Statement and Scope

Automotive retail across Tier-1 and Tier-2 Indian metropolitan markets has historically suffered from operational fragmentation. Prospective buyers face disjointed digital touchpoints, opaque on-road pricing estimates that omit local taxes, frequent double-booking of demonstrator fleet vehicles, and unverified pre-owned vehicle quality claims. On the dealership side, sales consultants struggle with scattered customer enquiries across third-party auto portals (such as CarDekho and CarWale), slow response times that degrade conversion rates, and a lack of auditability when adjusting certified pre-owned inventory pricing.

2.1 Indian Regulatory and Fiscal Framework

To reflect realistic commercial operations in Lucknow, Uttar Pradesh, the platform architecture incorporates two critical Indian legal and fiscal frameworks:

- **Digital Personal Data Protection Act 2023 (DPDP Act):** Mandates explicit, auditable customer consent prior to collecting personal identifiers (phone numbers, email addresses, and physical addresses). The platform enforces consent capture across all public enquiry and valuation forms, stores immutable consent timestamps, and masks vehicle registration numbers (e.g., UP32-XX-1234) on public pre-owned listings.
- **Indian Automotive Tax and Fiscal Architecture:** On-road pricing incorporates Ex-showroom base prices, Goods and Services Tax (GST) with applicable Compensation Cess, Uttar Pradesh State Road Transport Office (RTO) registration fees (8% for petrol/EV, 10% for diesel), mandatory multi-year insurance estimates, dealer logistics charges, and statutory Tax Collected at Source (TCS) of 1% on transactions exceeding ₹10,00,000.

2.2 Implemented Scope vs. Simulated Boundaries

To maintain rigorous architectural focus without unnecessary commercial dependencies, the platform explicitly defines what is implemented as working application logic versus what is simulated:

- **Fully Implemented Systems:** Public retail web SPA, authenticated staff administration portal, PostgreSQL relational schema with Prisma 6 ORM, concurrency-safe test-drive and workshop booking engine, lead CRM with automated lowest-workload consultant routing, 30-minute first-response SLA tracking, CSV batch ingestion with row-level validation, two-person used car pricing governance, and deterministic grounded domain intelligence.
- **Simulated Integration Boundaries:** Payment gateway checkout is represented via tokenized booking references rather than live payment gateway integration; telecom message delivery (WhatsApp/SMS) generates grounded, personalized message drafts for staff review rather than transmitting live traffic over commercial telecom aggregator APIs.

3. Technology Stack and Architecture Decisions

The Aurelia Motors platform is structured as an npm workspaces monorepo containing three coordinated packages: @aurelia/shared (isomorphic domain logic, Zod validation schemas, pricing calculators, and TypeScript interfaces), @aurelia/api (Express 5 backend, Prisma 6 ORM, and REST endpoints), and @aurelia/web (React 19 SPA frontend).

3.1 Component Architecture and Technology Selection

- **Frontend SPA (React 19, TypeScript, Vite, Tailwind CSS 4):** React 19 provides modern concurrent rendering and robust state primitives. Vite delivers fast hot module replacement. Tailwind CSS 4 provides a modern, utility-first design system without CSS runtime overhead. React Router 7 manages declarative client routing and route protection. TanStack Query v5 manages asynchronous server state, optimistic mutations, and background caching.
- **Backend REST API (Express 5, Node.js >= 22, Zod):** Express 5 provides an HTTP routing pipeline with native async error handling. Node.js >= 22.0.0 (supporting active LTS 22 and 24, aligned with node:22-alpine container images) ensures runtime stability. Zod enforces runtime boundary validation, sharing schema definitions between client and server to guarantee end-to-end type safety.
- **Data Persistence (PostgreSQL 18 Alpine / 16+, Prisma 6 ORM):** PostgreSQL provides ACID guarantees, row-level locking, and transactional consistency required to serialize booking requests. Prisma 6.19 generates type-safe database queries and manages declarative schema migrations across 36 models.

- **Security & Authentication (Argon2id, Session Cookies, CSRF):** Staff authentication utilizes Argon2id password hashing, signed HttpOnly SameSite=Lax session cookies, double-submit CSRF tokens, and location-scoped Role-Based Access Control (RBAC) middleware.

3.2 Key Architectural Decisions and Trade-Offs

- **Integer Paise Currency Standard:** Floating-point numbers in JavaScript and SQL introduce precision loss during tax and EMI calculations. All financial values (ex-showroom, RTO, insurance, TCS, discounts, and trade-in valuations) are strictly represented as 64-bit BigInt integer paise (1 INR = 100 paise). Conversion to rupee display strings occurs only at the presentation boundary.
- **Deterministic Grounded Domain Engine vs. External Cloud LLM:** Automotive retail requires strict factual accuracy regarding stock availability, vehicle specifications, and pricing. Relying on external cloud LLMs introduces unpredictable latency, recurring API costs, external network failure modes, and generative confabulation risks. Aurelia Motors implements a deterministic, heuristic-driven domain engine directly grounded in live PostgreSQL data, delivering fast local responses without generative errors or cloud API dependencies.

4. System Architecture Diagram

The system employs a containerized multi-tier architecture isolating edge routing, application processing, and data persistence:

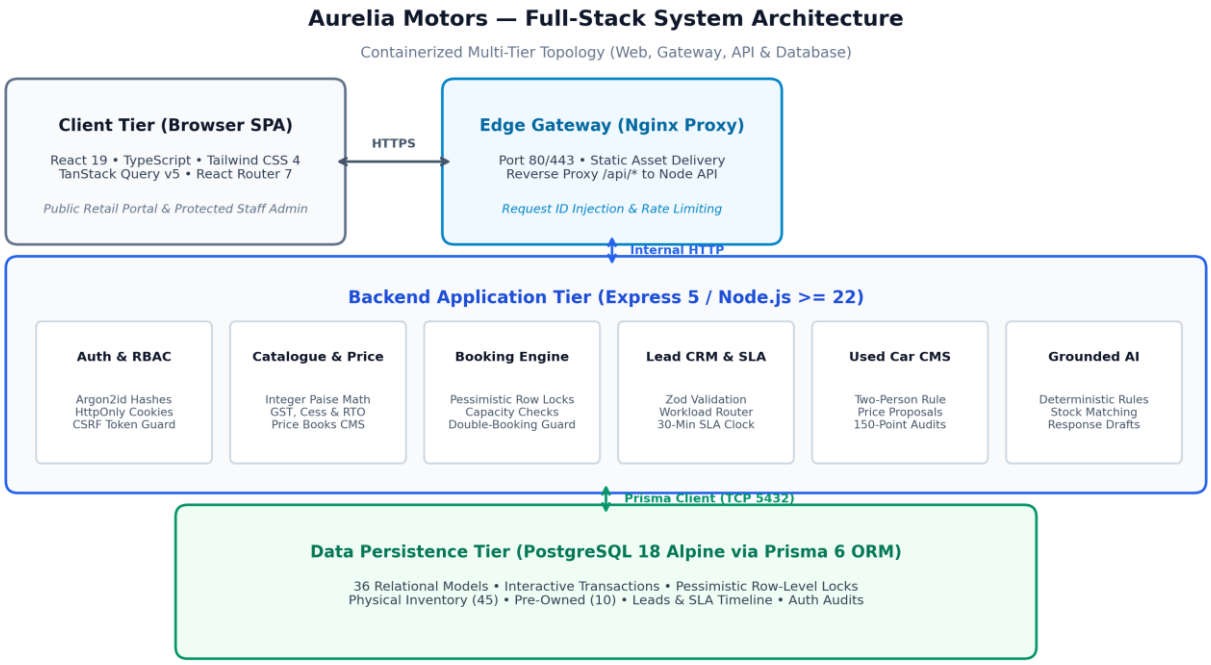


Figure 4.1: Aurelia Motors End-to-End System Architecture and Container Topology

System Request Lifecycle Walkthrough:

- **1. Ingress & Edge Proxy Tier:** Incoming HTTP/HTTPS client requests hit the Nginx edge proxy container on port 80/443. Nginx serves production-built static assets for the React 19 SPA, injects unique X-Request-ID headers, enforces client request rate limits, and reverse-proxies all /api/* traffic to the Express API container.
- **2. Application Middleware Pipeline:** Express 5 parses incoming JSON payloads and cookies, evaluates CORS policies, logs request traces via Pino, and executes CSRF protection middleware on mutating HTTP methods (POST, PATCH, DELETE, PUT). Authentication middleware decrypts and validates signed session cookies, hydrating the staff security context.
- **3. Route Controllers & Zod Validation:** Requests pass through modular controllers where input schemas are parsed using isomorphic Zod schemas. Invalid payloads are rejected immediately with structured HTTP 422 Unprocessable Entity envelopes containing exact field-level errors.
- **4. Domain Services & Concurrency Controls:** Business logic executes within specialized domain services. For booking operations, services wrap mutations in interactive Prisma transactions with pessimistic row-level locking (SELECT ... FOR UPDATE) to serialize concurrent slot requests.

- **5. Data Access & Persistence Tier:** Prisma Client executes parameterized SQL queries against PostgreSQL over dedicated connection pools, returning strongly typed models to the application tier.

5. Database Design and Entity Relationships

The relational data model is normalized into 36 tables designed to model dealership facilities, vehicle catalogs, physical inventory units, certified pre-owned cars, pricing books, staff authorization, leads, appointments, and audit trails.

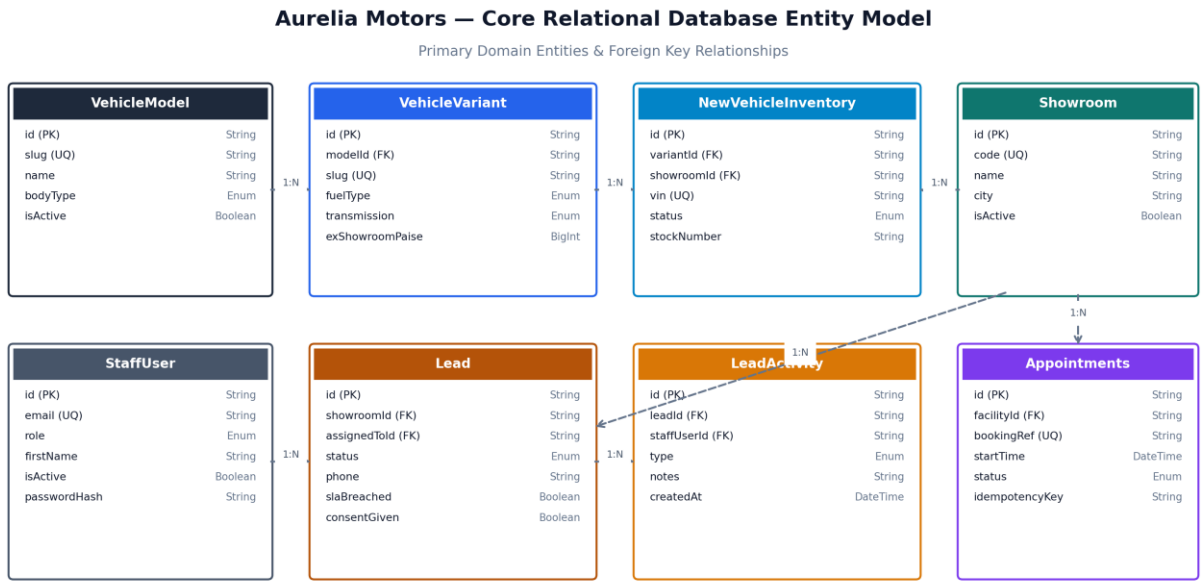


Figure 5.1: Simplified Core Relational Database Entity-Relationship Model

5.1 Database Entity Model Catalog

The primary database models and their seeded record counts are summarized below:

Prisma Model	Primary Key	Key Foreign Keys	Functional Responsibility	Seeded Rows
Showroom	id (CUID)	None	Physical retail sales facilities across Lucknow	3 facilities
Workshop	id (CUID)	None	Authorized vehicle maintenance workshops with bay limits	2 centers
VehicleModel	id (CUID)	None	Passenger vehicle models (Lumen, Meridian, Solace, etc.)	6 models
VehicleVariant	id (CUID)	modelId	Powertrain trims, transmissions, specifications, base prices	15 variants
NewVehicleInventory	id (CUID)	variantId, showroomId	Physical showroom inventory units with VIN and stock status	45 units
UsedVehicle	id (CUID)	showroomId	Certified pre-owned cars with masked plates & inspection score	10 units
UsedVehicleInspection	id (CUID)	usedVehicleId	Comprehensive 150-point mechanical & electrical inspection	10 audits
PriceBook	id (CUID)	None	Versioned pricing schemes with effective dates and overlap locks	2 price books
PriceComponent	id (CUID)	priceEntryId	Configurable fee/tax components (FIXED or PERCENT_OF_EXSHOWROOM)	Seed components
StaffUser	id (CUID)	None	Staff credentials, Argon2id hashes, and RBAC operational roles	4 accounts
Lead	id (CUID)	showroomId, assignedStaffId	Inbound customer enquiries with 30-min SLA tracking clock	Dynamic

Prisma Model	Primary Key	Key Foreign Keys	Functional Responsibility	Seeded Rows
TestDriveAppointment	id (CUID)	showroomId, variantId	Capacity-controlled demonstrator vehicle booking slots	Dynamic
ServiceAppointment	id (CUID)	workshopId	Daily bay capacity-controlled workshop service reservations	Dynamic
AuthenticationAuditEvent	id (CUID)	staffUserId	Immutable chronological audit log of staff authentication events	Dynamic

6. Core Customer Workflows

The public retail application delivers self-service automotive retail capabilities across five core customer workflows:

6.1 Vehicle Discovery and Multi-Variant Comparison

Prospective buyers explore the active vehicle catalogue across six passenger models (Lumen, Meridian, Solace, Kestrel, Zephyr, and Apex). Users filter catalogue results by body type (Sedan, Compact SUV, Mid-Size SUV, Hatchback), fuel type (Petrol, Diesel, Electric, Hybrid), transmission (Manual, Automatic, DCT, e-CVT), and budget ranges. The interface supports side-by-side technical specification comparisons (dimensions, engine output, boot volume, safety ratings, and fuel efficiency) across variants.

6.2 On-Road Price Transparency and Financing Calculations

Aurelia Motors provides an itemized on-road price calculation modeled on Uttar Pradesh RTO regulations and Indian tax rules. Components are stored as configurable price entries calculated via basis points or fixed integer paise.

On-Road Pricing Calculation Formula
On-Road Price = Ex-Showroom + RTO Registration + Insurance + TCS + Handling/Logistics + Optional Accessories

Where:
• Ex-Showroom: Versioned base price in integer paise defined in the active showroom price book.
• RTO Registration: Configurable percentage rate (e.g., 8% for Petrol/EV; 10% for Diesel in Uttar Pradesh).
• Comprehensive Insurance: Mandatory 1-year Own Damage + 3-year Third Party policy (bracketed base + percentage rate).
• Tax Collected at Source (TCS): Statutory 1% under Section 206C(1F) of the Income Tax Act on any vehicle exceeding ₹10,00,000 (100,000,000 paise).
• Logistics & Handling: Configurable dealer administrative fee (sample: ₹15,000).

The on-road pricing engine is paired with an interactive reducing-balance EMI calculator. Buyers adjust loan principal, tenure (12 to 84 months), and annual interest rates (typically 8.5% to 11.5% p.a.). The monthly repayment is computed via:

Reducing-Balance Equated Monthly Installment (EMI) Formula
EMI = P * r * (1 + r)^n / ((1 + r)^n - 1)

Where:
• P: Principal loan amount in integer paise (Ex-Showroom minus customer down-payment).
• r: Monthly interest rate (Annual Interest Rate / 12 / 100).
• n: Total loan tenure in months (e.g., 60 months for a 5-year loan).

6.3 Capacity-Safe Test-Drive Reservations

Test-drive bookings validate against the physical demonstrator fleet assigned to each showroom. To serialize concurrent booking requests, the booking service executes inside an interactive Prisma transaction featuring explicit pessimistic row-level locking on assigned demonstrator vehicle records:

```
// Concurrency-Safe Demonstrator Booking with Pessimistic Row Lock
return await prisma.$transaction(async (tx) => {
  const existing = await tx.testDriveAppointment.findUnique({ where: { idempotencyKey } });
  if (existing) return existing;
```

```
  // Pessimistic lock on showroom demo fleet to serialize concurrent bookings
  await tx.$queryRaw`SELECT id FROM demo_vehicles WHERE id = ANY(${vehicleIds}) FOR UPDATE`;
  const booked = await tx.testDriveAppointment.findMany({ where: { demoVehicleId: { in: vehicleIds },
... } });
  const freeVehicle = demoVehicles.find((v) => !bookedIds.has(v.id));
  if (!freeVehicle) throw new AppError(409, 'SLOT_UNAVAILABLE', 'Selected slot is fully booked');
  return tx.testDriveAppointment.create({ data: { bookingReference, demoVehicleId: freeVehicle.id,
... } });
}, { isolationLevel: 'ReadCommitted' });
```

Upon successful reservation, the customer receives an alphanumeric booking reference (e.g., TD-2026-8742) and an immutable confirmation token. Customers can self-service cancel or view appointments using this reference and token.

6.4 Workshop Service Appointment Scheduling

Vehicle maintenance appointments are scheduled across the two authorized Lucknow workshops (Transport Nagar and Alambagh). The scheduling engine enforces daily capacity limits (maximum 8 service appointments per workshop per calendar day) using pessimistic locks on the workshop row (`SELECT id FROM workshops WHERE id = ... FOR UPDATE`) inside an interactive transaction. Customers select from standardized service packages (Periodic Maintenance, Monsoon Clinic, Brake Overhaul, and Diagnostic Audit).

6.5 Algorithmic Pre-Owned Trade-In Valuation

Prospective buyers seeking to trade in an existing vehicle receive an instant valuation range without registering. The valuation algorithm considers base original invoice price, annual straight-line depreciation (12% per year), mileage penalties (₹1.50 per km above 10,000 km/year threshold), and condition adjustment multipliers (0.90 for Fair, 1.00 for Good, 1.08 for Excellent). The engine outputs an indicative lower/upper valuation band.

7. Dealership Operations

Internal dealership operations automate lead qualification, sales routing, response tracking, catalog CMS, and pricing governance:

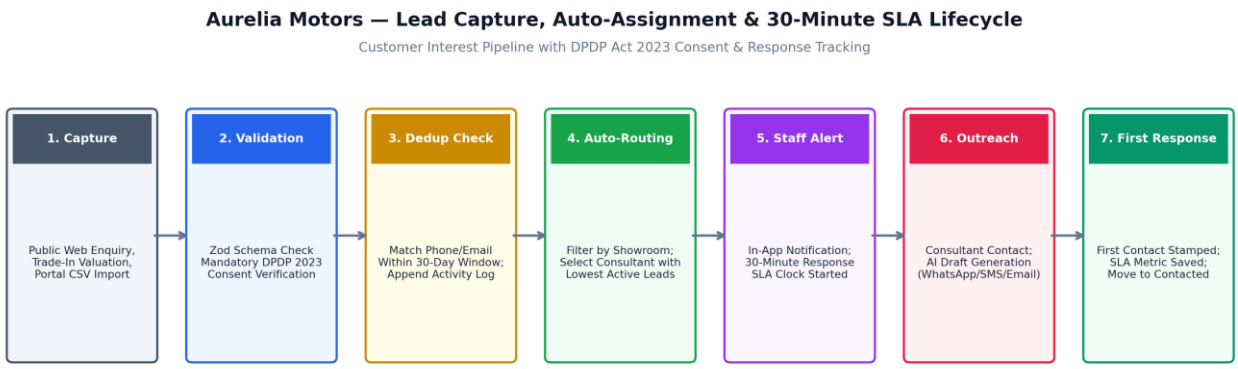


Figure 7.1: Inbound Lead Processing, Workload Auto-Assignment & 30-Minute SLA Lifecycle

7.1 Inbound Lead Capture and Automated Routing

Customer enquiries originate from public web forms, trade-in valuations, and third-party portal batch CSV imports. The lead processing engine executes in five automated stages:

- **1. Consent & Zod Validation:** Enforces mandatory DPDP Act 2023 customer consent. Rejects invalid phone numbers or missing facility identifiers.
- **2. Duplicate Detection:** Checks for active leads with matching phone or email within the preceding 30 days. If found, appends the new interest to the existing customer timeline rather than creating fragmented duplicate leads.
- **3. Workload-Balanced Routing:** Filters sales consultants assigned to the target showroom (or USED_CAR_STAFF for trade-ins). Selects the consultant with the lowest count of active leads in statuses [NEW, ASSIGNED, CONTACTED, QUALIFIED, NEGOTIATING]. Ties are broken deterministically by least recently assigned timestamp.

- **4. SLA Clock Initialization:** Initializes a 30-minute first-response SLA timer. Dispatches an in-app notification to the assigned consultant's dashboard.

7.2 Lead CRM Pipeline and 30-Minute SLA Tracking

Responding to customer inquiries promptly significantly increases conversion rates. The staff CRM inbox highlights active leads with real-time countdown timers. When a consultant logs an outreach activity (Phone Call, WhatsApp Message, Email, or In-Person Showroom Visit), the system records the exact response timestamp. If the response exceeds 30 minutes, the lead is permanently tagged as SLA breached for management reporting.

7.3 External Portal CSV Batch Ingestion

Dealerships receive prospective leads exported from external portals (e.g., CarDekho and CarWale). Aurelia Motors provides a two-stage CSV ingestion engine:

- **Stage 1: Validation Preview:** POST /api/staff/leads/import/preview parses uploaded CSV rows, validates phone numbers and model slugs, detects duplicate customer records, and returns structured summary metrics prior to database mutation.
- **Stage 2: Atomic Batch Ingestion:** POST /api/staff/leads/import executes inside a database transaction, batch-inserting qualified leads, automatically assigning consultants based on current showroom workloads, and creating audit logs for the import batch.

7.4 Inventory CMS and Price Book Governance

Dealership administrators manage the vehicle lifecycle via protected CMS endpoints: provisioning new vehicle models, configuring variant powertrain parameters, updating showroom stock allocations, and publishing versioned price books. Price books enforce non-overlapping effective date intervals, preventing conflicting retail prices.

7.5 Used-Car Pricing Proposals and Separation of Duties

To maintain margin discipline, certified pre-owned price adjustments enforce a strict separation of duties:

- **Price Adjustment Proposal:** A Used Car Specialist inspects a certified vehicle and submits a price adjustment proposal (POST /api/staff/used-vehicles/:id/propose-price) in integer paise with written commercial justification.
- **Administrative Decision & Separation of Duties:** An Administrator reviews pending proposals (POST /api/staff/used-car-pricing/proposals/:id/approve). The system enforces separation of duties via role-based access control and an application-level identity check: the approving administrator cannot be the staff member who authored the proposal (proposal.proposedByStaffId === reviewer.id).

8. AI-Assisted Features and Limitations

Aurelia Motors incorporates the Aurelia Grounded Domain Engine—a deterministic, rules-based intelligence system grounded in live PostgreSQL catalog and inventory state. The engine provides three specialized staff-assistance capabilities:

8.1 Implemented Domain Intelligence Capabilities

- **1. Structured Lead Intent & Urgency Scoring (POST /api/staff/ai/:leadId/summary):** Parses enquiry notes, identifies customer powertrain preferences, extracts stated budget parameters, and computes an objective urgency score (1 to 5) based on purchase timeframe keywords (e.g., 'immediate delivery', 'this weekend').
- **2. Human-in-the-Loop Multi-Channel Response Drafts (POST /api/staff/ai/:leadId/response-draft):** Generates tailored customer communication drafts across WhatsApp, SMS, and Email channels. Drafts dynamically inject customer name, requested vehicle variant, showroom address, assigned consultant contact details, and test-drive invitation links. Drafts are presented to sales consultants for review and manual dispatch, ensuring full human-in-the-loop oversight.
- **3. Inventory-Grounded Vehicle Recommendations (POST /api/staff/ai/:leadId/recommendations):** Queries live PostgreSQL tables to identify physical inventory units and certified pre-owned cars matching the customer's budget bracket (+/- 15%) and powertrain preferences. Strictly filters against units currently marked as AVAILABLE in the database.

8.2 Explicit Boundaries and Limitations

The AI domain engine adheres to strict engineering guardrails:

- **Elimination of Generative Confabulation:** Because responses are derived deterministically from database queries and structured templates rather than probabilistic cloud LLMs, the engine does not invent non-existent models or stock.

- **No Automated External Transmission:** The engine generates message drafts for consultant review. It does not dispatch live telecom SMS or WhatsApp messages without human confirmation.
- **Fast Local Execution:** Executes in-process on the local Node.js runtime without external cloud API calls, avoiding third-party network latency and recurring API billing.

9. Authentication, Authorization and Data Privacy

The security architecture enforces defense-in-depth across credentials, session management, access control, and privacy compliance:

9.1 Credential Security and Session Management

- **Password Hashing:** Staff passwords are hashed using Argon2id with secure random salts (minimum 8 characters, requiring uppercase, lowercase, numbers, and symbols).
- **HttpOnly Session Cookies:** Sessions are maintained via signed, HttpOnly, SameSite=Lax cookies, mitigating Cross-Site Scripting (XSS) session theft.
- **CSRF Defense:** Double-submit CSRF cookie and header validation is enforced across all mutating staff endpoints (POST, PATCH, DELETE, PUT).
- **Brute-Force Rate Limiting:** Login endpoints enforce rate limits (max 5 failed attempts per 15-minute window) using Express rate-limiting middleware.

9.2 Role-Based Access Control (RBAC) Matrix

The system defines four discrete operational roles with strictly enforced permission boundaries:

Operational Role	Location Scope	Leads & CRM	Bookings & Service	Inventory & CMS	Staff Admin
ADMIN	All Facilities	Full View & Reassign	Full View & Manage	Create, Edit & Publish	Full Staff Admin
SALES_CONSULTANT	Assigned Showrooms	Assigned Leads Only	Showroom Test-Drives	Read-Only Inventory	No Access
USED_CAR_STAFF	Assigned Showrooms	Trade-In Enquiries	No Access	Propose Used Prices	No Access
SERVICE_ADVISOR	Assigned Workshops	No Access	Workshop Bay Queue	No Access	No Access

9.3 Data Privacy & DPDP Act 2023 Compliance

- **Explicit Consent Auditing:** Customer enquiry forms require an affirmative checkbox consenting to contact under the Digital Personal Data Protection Act 2023. The CustomerConsent table records consent timestamps, consent text versions, and client IP hashes.
- **Plate Masking on Pre-Owned Vehicles:** Public certified pre-owned listings mask vehicle registration numbers (e.g., displaying UP32-XX-1234 instead of full registration) to protect previous owners' privacy until formal sales negotiation.

10. Testing and Verification

Aurelia Motors employs a multi-tiered automated test suite executed via Vitest and Supertest. The automated test suite comprises 127 automated test assertions across 18 dedicated test files:

10.1 Multi-Tiered Testing Strategy and Scope

- **Tier 1: Domain Unit Tests (@aurelia/shared — 3 suites, 14 tests):** Validates mathematical calculations: on-road price tax itemization, reducing-balance EMI loan calculations, used vehicle depreciation algorithms, and Indian currency/number formatting.
- **Tier 2: API & Integration Tests (@aurelia/api — 14 suites, 99 tests):** Executes Supertest HTTP assertions against real Express routes and PostgreSQL, verifying Zod schema boundary validation, Prisma database queries, Argon2id authentication, session cookies, CSRF tokens, and RBAC authorization.
- **Tier 3: End-to-End System Journey Tests (@aurelia/api — 1 suite, 14 tests):** e2e-journeys.test.ts executes multi-step integration journey assertions via Supertest against the running Express HTTP pipeline and PostgreSQL database: Model Discovery to Lead Assignment, Certified Pre-Owned Inspection to Trade-In Valuation, Test-Drive Double-Booking Prevention, Workshop Bay Capacity Limits, and Staff CRM Operations. Note on Test Scope: The repository does not

include browser-driven UI automation (such as Playwright or Cypress); all automated test assertions operate at the shared domain logic and API integration levels.

10.2 Automated Test Suite Inventory

Package & Suite File	Test Scope & Focus	Test Count	Key Verification Assertions
@aurelia/shared/finance.test.ts	Domain Unit Math	6 tests	Reducing-balance EMI formulas, edge cases (0% interest, 84-mo tenures)
@aurelia/shared/pricing.test.ts	Pricing Domain	4 tests	Integer paise math, GST, RTO tiers (8% petrol vs 10% diesel), 1% TCS
@aurelia/shared/format.test.ts	Formatting Utilities	4 tests	Lakh/Crore Indian numbering, phone masking, date-time representations
@aurelia/api/validation.test.ts	Zod Validation	2 tests	Accepts valid payload bodies; returns structured 422 errors for malformed inputs
@aurelia/api/health.test.ts	Service Health	7 tests	Health envelope, database connectivity probe, request ID tracking, CORS
@aurelia/api/showrooms.test.ts	Facility Discovery	3 tests	Active Lucknow showrooms, facility detail by code, 404 error boundaries
@aurelia/api/catalogue.test.ts	Vehicle Catalogue	6 tests	Model listings, specification filters, transparent showroom on-road pricing
@aurelia/api/used-vehicles.test.ts	Certified Pre-Owned	6 tests	Masked plate browsing, 150-point inspection findings, sorting by price/km
@aurelia/api/appointments.test.ts	Test-Drive & Service	8 tests	Slot availability, double-booking rejection, workshop bay limits, cancellation
@aurelia/api/auth.test.ts	Auth & RBAC Security	10 tests	Argon2id login, HttpOnly cookies, CSRF defense, role-based route blocking
@aurelia/api/leads.test.ts	Lead CRM & Ingestion	14 tests	DPDP consent enforcement, 30-min SLA clock, auto-routing, CSV batch ingestion
@aurelia/api/cms.test.ts	Admin CMS Governance	7 tests	Model/variant provisioning, inventory stock updates, price book overlap locks
@aurelia/api/used-car-pricing.test.ts	Pricing Governance	6 tests	Price proposals, separation of duties, self-approval prevention (403)
@aurelia/api/ai.test.ts	Grounded Domain AI	6 tests	Deterministic lead intent summaries, multi-channel response drafts, stock match
@aurelia/api/notifications.test.ts	Staff Notifications	5 tests	In-app alerts, unread counts, cross-staff ownership guards
@aurelia/api/content.test.ts	CMS Promotions	4 tests	Homepage promotional banners, finance schemes, admin mutation guards
@aurelia/api/trade-ins.test.ts	Trade-In Valuations	5 tests	Algorithmic valuation bands, DPDP consent, reference tracking
@aurelia/api/e2e-journeys.test.ts	API Journey Tests	14 tests	Multi-step customer and staff lifecycle integration journeys via Supertest

11. Local Setup and Deployment

Aurelia Motors supports two deployment modes: local multi-workspace development via npm and containerized Docker Compose.

11.1 System Prerequisites

- **Node.js Runtime:** Node.js >= 22.0.0 (LTS 22 or 24) and npm >= 10.0.0 (local development host or container).
- **Database:** PostgreSQL 16+ or Docker Desktop running postgres:18-alpine.
- **Container Engine:** Docker Desktop with Compose v2 support (optional for containerized mode).

11.2 Local Development Quickstart

```
# 1. Clone repository and install dependencies
git clone https://github.com/lochan87/aurelia-motors.git
```

```
cd aurelia-motors
npm install

# 2. Build shared package and initialize database
npm run build:shared
npm run db:migrate --workspace @aurelia/api
npm run db:seed --workspace @aurelia/api

# 3. Start development servers
npm run dev
```

The public web application is accessible at http://localhost:5173, and the Express REST API at http://localhost:4000.

11.3 Containerized Docker Compose Deployment

The application includes production Dockerfiles for each tier and a coordinated docker-compose.yml configuration:

```
# Start all container services
docker compose up -d --build

# Verify container health
curl http://localhost/api/health
```

Docker Container Topology:

- **aurelia-web:** Nginx Alpine reverse proxy on host port 80, serving React 19 production assets.
- **aurelia-api:** Express 5 Node.js container (node:22-alpine) on host port 4000 (proxied via Nginx on /api/*).
- **aurelia-postgres:** PostgreSQL 18 Alpine container (postgres:18-alpine) on host port 55432 (mapped to internal port 5432).

11.4 Seed Demonstration Accounts

The database seed provisions four demonstration accounts representing distinct operational roles:

Staff Email	Operational Role	Assigned Facility	Demonstration Scope
admin@aureliamotors.com	ADMIN	All Facilities	Full dealership administration, CMS, price approvals, staff user management
sales.gomti@aureliamotors.com	SALES_CONSULTANT	Gomti Nagar Showroom	Showroom lead inbox, 30-min SLA response tracking, test-drive schedules
usedcars@aureliamotors.com	USED_CAR_STAFF	Central Showroom	Certified pre-owned inspections, trade-in valuations, price adjustment proposals
service.transport@aureliamotors.com	SERVICE_ADVISOR	Transport Nagar Workshop	Workshop bay appointment management, vehicle check-ins, service status

Note on Demonstration Credentials: In accordance with security best practices, plaintext passwords are not published in this technical specification. Demonstration staff accounts are provisioned with environment-configurable credentials documented in deployment.md (default: AURELIA_DEMO_PASSWORD). No production credentials exist.

12. Known Limitations and Future Improvements

In accordance with transparent engineering principles, the current architecture identifies explicit operational boundaries and outlines the production hardening roadmap:

12.1 Known Limitations

- **Simulated Gateway Integration:** Booking deposits and vehicle token reservations generate cryptographic confirmation references rather than processing actual credit card or UPI transactions through a production payment gateway (such as Razorpay or Stripe).

- **Simulated Telecom Outbound Dispatch:** Customer notification drafts (WhatsApp, SMS, Email) are generated deterministically and formatted for human review, but are not automatically transmitted over commercial telecom aggregator APIs.
- **Database-Level Concurrency Locks:** Capacity checks and double-booking guards leverage PostgreSQL interactive transactions and row-level locks. While robust for single-node multi-facility deployments, extreme high-throughput scale would benefit from distributed Redis locks.

12.2 Production Hardening Roadmap

- **Real-Time WebSocket / SSE Push:** Upgrade staff notification and lead SLA countdown polling intervals to native Server-Sent Events (SSE) or WebSockets for zero-latency inbox alerts.
- **External Automotive Inventory Feeds:** Implement automated DMS (Dealer Management System) sync protocols (DMS-Open, CDK, or Reynolds & Reynolds) for automated synchronization of factory vehicle dispatches and yard inventory.
- **Multi-Showroom Read Replicas:** Deploy PostgreSQL read replicas across AWS ap-south-1 (Mumbai) and ap-south-2 (Hyderabad) regions to serve public vehicle browsing queries with low latency.

13. Repository and Reviewer Access

The Aurelia Motors platform source code, automated test suites, deployment manifests, and technical documentation are version-controlled and accessible via the official GitHub repository:

- **Repository URL:** https://github.com/lochan87/aurelia-motors
- **Primary Branch:** main
- **Author:** Lochan T N
- **License:** No license file; submitted for hiring evaluation

13.1 Reviewer Verification Commands

```
# Execute complete test suite (127 automated tests across 18 suites)
npm test

# Execute code formatting & lint checks
npm run lint

# Verify TypeScript types across all workspaces
npm run typecheck
```

For comprehensive step-by-step local setup, Docker Compose deployment instructions, and troubleshooting procedures, please refer to deployment.md in the root directory.